

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № 1
по дисциплине «Структуры и алгоритмы обработки данных»
по теме: «Анализ временной сложности элементарных алгоритмов»

Выполнил студент группы:

БВТ2505

Соронина Арина Андреевна

Проверил: преподаватель Лапин

Виктор Андреевич

Москва 2026

Оглавление

Выполнение.....	4
Вывод.....	15

Цель и задачи работы:

Цель данной лабораторной работы: научиться определять временную сложность алгоритма аналитически и подтверждать оценку экспериментально.

Задачи:

1. Сгенерировать данные своего варианта: `python scripts/generate_data.py --variant N --only arrays` (пять размеров с геометрическим шагом, $10^3 \dots 10^5$, четыре типа массивов). Данные детерминированы: `seed = 30 + N`.
2. Реализовать четыре элементарных алгоритма: (а) сумма элементов массива; (б) поиск максимума; (в) подсчёт пар равных элементов (двойной цикл); (г) бинарное возведение в степень (с поддержкой вычислений по модулю).
3. Для каждого алгоритма **аналитически** вывести асимптотическую оценку времени (худший случай) и записать её в отчёте с обоснованием.
4. Выполнить замеры времени на данных своего варианта по методике бенчмаркинга (прогрев, ≥ 5 повторов, медиана) и построить графики: $\log\text{-}\log$ для степенных алгоритмов и $t(\log_2 n)$ для бинарного возведения в степень.
5. Сопоставить наблюдаемый наклон графика с теоретической оценкой; расхождения объяснить. Пояснить, почему для логарифмического алгоритма $\log\text{-}\log$ график неинформативен и почему замер выполняется по модулю (длинная арифметика Python).
6. Проверить реализации на репрезентативных тестах (пустой массив, один элемент, все элементы равны) и зафиксировать инварианты; сверить результаты с эталонными реализациями (`sum`, `max`, трёхаргументный `pow`).

Выполнение

1. Реализация четырёх элементарных алгоритмов и вывод асимптотической оценки времени

1. Сумма элементов массива.

```
def array_sum(a: list[int]) -> int:
    s = 0
    for x in a:
        s += x
    return s
```

Один проход, константная работа на итерации: $T(n) = c_1 + c_2 \cdot n = \Theta(n)$. Лучший, средний и худший случаи совпадают: массив всегда просматривается целиком, поэтому оценка верна и сверху, и снизу – отсюда Θ .

2. Максимум массива (массив непуст).

```
def array_max(a: list[int]) -> int:
    s = a[0]
    for x in a:
        if x > s:
            s = x
    return s
```

Один проход, на каждой итерации сравнение и в худшем случае присваивание – обе операции $O(1)$: $T(n) = c_1 + c_2 \cdot (n - 1) = \Theta(n)$. Цикл нельзя остановить раньше: чтобы убедиться, что среди оставшихся элементов нет большего, нужно проверить их все, поэтому лучший, средний и худшие случаи совпадают – оценка верна и сверху, и снизу, отсюда Θ .

3. Число пар (i, j) , $i < j$, таких что $a[i] == a[j]$.

```
def count_equal_pairs(a: list[int]) -> int:
    k = 0
    n = len(a)
    for i in range(n):
        for j in range(i + 1, n):
            if a[i] == a[j]:
                k += 1
    return k
```

Двойной цикл: внешний проход по i от 0 до $n-1$, внутренний по j от $j+1$ до $n-1$, на каждой паре одно сравнение $O(1)$. Число сравнений – сумма $(n + 1) + (n - 2) + \dots + 1 + 0 = n(n-1)/2$, откуда $T(n) = c \cdot n(n - 1)/2 = \Theta(n^2)$. Сравнение $a[i] == a[j]$ выполняется для каждой пары независимо от значений элементов – от данных зависит только счётчик совпадений, а не число самих сравнений, поэтому лучший, средний и худший случаи совпадают – оценка верна и сверху, и снизу — отсюда Θ .

4. Бинарное возведение в степень, $n \geq 0$. При заданном mod все умножения выполняются по модулю (результат $x^{**}n \% \text{mod}$).

```
def binary_pow(x: int, n: int, mod: int | None = None) -> int:
    r = 1
    while n > 0:
        if n % 2 == 1:
            r *= x
            if mod is not None:
                r %= mod
        x *= x
        if mod is not None:
            x %= mod
        n //= 2
    return r
```

Цикл делит n пополам на каждой итерации ($n //= 2$), число итераций – $\lfloor \log_2 n \rfloor + 1$, на каждой из них проверка бита, умножение и взятие по модулю – все $O(1)$. Отсюда $T(n) = c \cdot (\lfloor \log_2 n \rfloor + 1) = \Theta(\log n)$. Число итераций зависит только от величины n , а не от x – условное умножение $r^* = x$ – ветвление, но обе стоят $O(1)$ и на асимптотику не влияют, поэтому лучший, средний и худший случаи совпадают – оценка верна и сверху, и снизу, отсюда Θ .

2. Замеры времени на данных своего варианта по методике бенчмаркинга

```
self_check: OK

array_sum:
  n= 1000 t=0.000025 c
  n= 3000 t=0.000067 c
  n= 10000 t=0.000267 c
  n= 30000 t=0.000765 c
  n= 100000 t=0.002581 c

array_max:
  n= 1000 t=0.000017 c
  n= 3000 t=0.000050 c
  n= 10000 t=0.000169 c
  n= 30000 t=0.000491 c
  n= 100000 t=0.001638 c

count_equal_pairs:
  n= 500 t=0.005376 c
  n= 1000 t=0.022085 c
  n= 3000 t=0.210430 c
  n= 10000 t=1.992676 c
  n= 30000 t=39.693095 c

binary_pow (по 20000 вызовов на точку, по модулю 1000000007):
  n= 1000 log2(n)= 10.0 t=0.000001579 c
  n= 10000 log2(n)= 13.3 t=0.000002240 c
  n= 100000 log2(n)= 16.6 t=0.000002599 c
  n= 1000000 log2(n)= 19.9 t=0.000003084 c
  n= 10000000 log2(n)= 23.3 t=0.000003640 c
```

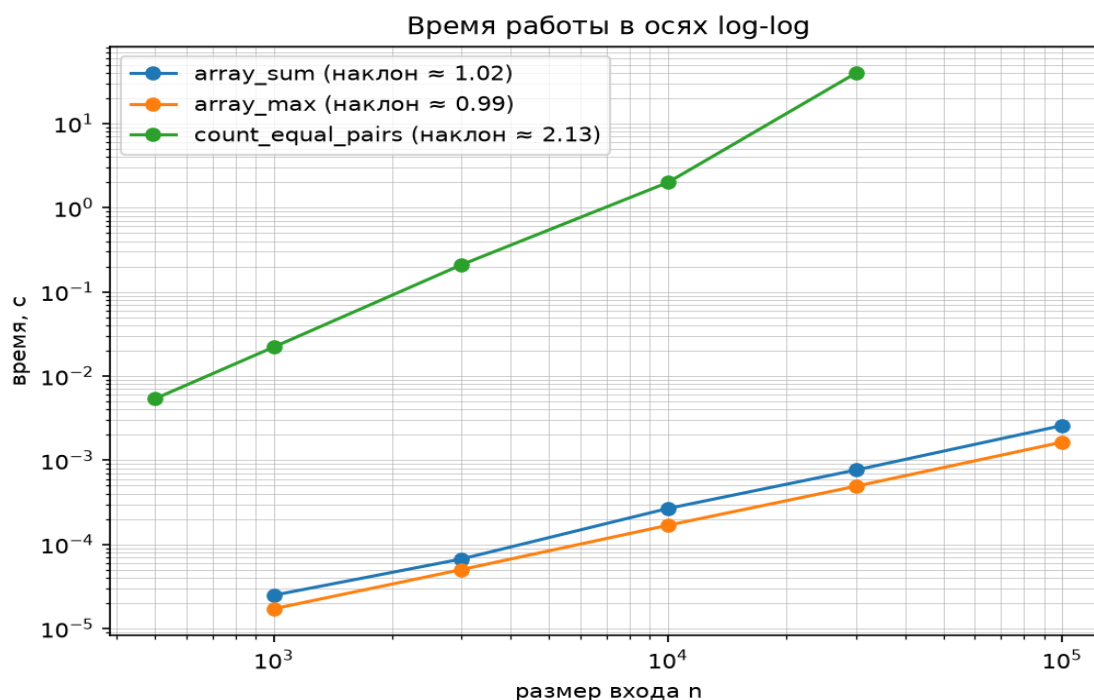
3. Сопоставление наблюдаемого наклона с теоретической оценкой

Технические характеристики:

CPU: 12th Gen Intel(R) Core(TM) i5-12450H

Python: version 3.13

1. Степенные алгоритмы (log-log график).



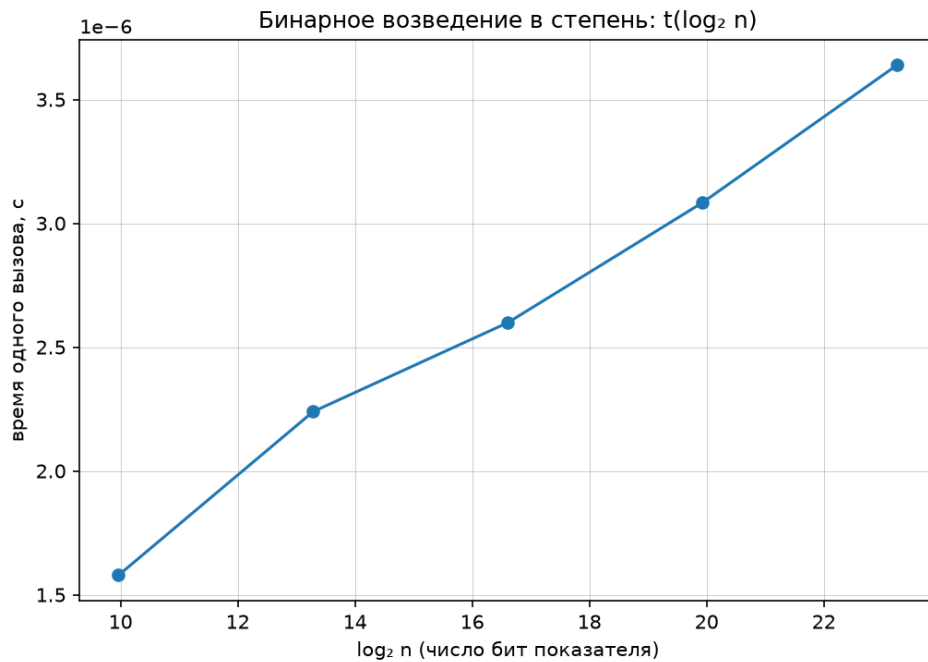
Для алгоритмов с полиномиальной сложностью $T(n) = c \cdot n^k$ логарифмирование обеих частей даёт $\log T = \log c + k \cdot \log n$ – прямую линию в осях log-log, угол наклона которой равен показателю степени k. Это позволяет напрямую сравнить эмпирический наклон с теоретическим значением k:

Алгоритм	Теоретическая оценка	Предсказанный наклон	Наблюдаемый наклон	Отклонение
Array_sum	$\Theta(n)$	1	1.02	~2%
Array_max	$\Theta(n)$	1	0.99	~1%
Count_equal_pairs	$\Theta(n^2)$	2	2.13	~6,5%

Для `array_sum` и `array_max` наблюдаемый наклон практически совпадает с теоретическим – отклонение укладывается в погрешность измерения. Для `count_equal_pairs` отклонение заметнее.

Объяснение расхождений. Небольшие отклонения (1–2%) у линейных алгоритмов объясняются шумом замера: конечным числом повторов, фоновой нагрузкой операционной системы и ограниченным разрешением таймера – методика (медиана из ≥ 5 повторов) снижает, но не устраняет этот шум полностью. Более заметное превышение теоретического наклона у `count_equal_pairs` (2.13 вместо 2.0) объясняется тем, что модель RAM считает любое обращение к памяти операцией одинаковой стоимости $O(1)$, в то время как в реальном процессоре это не так: при росте n рабочий набор данных, обрабатываемый вложенным циклом, перестаёт помещаться в кэш процессора, и обращения к оперативной памяти начинают обходиться дороже, чем обращения к кэшу. Поскольку этот эффект усиливается с ростом n , реальное время растёт немного быстрее теоретической оценки n^2 , что и даёт на графике слегка завышенный наклон. Дополнительный вклад вносит малое число точек измерения (5), из-за чего оценка наклона методом наименьших квадратов менее устойчива, чем при большем числе точек.

2. Логарифмический алгоритм (binary_pow).



Здесь наклон линии на графике $t(\log_2 n)$ равен $\sim 1.49 \cdot 10^{-7}$ с на один бит показателя степени (получено линейной регрессией по пяти точкам), при этом коэффициент детерминации $R^2 \sim 0.993$ – то есть точки ложатся на прямую практически без отклонений. Прямолинейность графика в этих координатах и является подтверждением оценки $\Theta(\log n)$: согласно теории, $T(n) = c \cdot (\lfloor \log_2 n \rfloor + 1)$, то есть T линейно зависит от $x = \log_2 n$, а не от n напрямую.

Почему для `binary_pow` `log-log` график неинформативен: `Log-log` график линеаризует именно степенные зависимости $T = c \cdot n^k$ – потому что логарифмирование превращает степень k в угловой коэффициент прямой. Логарифмическая же функция $T = c \cdot \log_2 n$ степенной не является: у неё нет показателя степени, который можно было бы «вытащить» через наклон. Если построить $\log(T)$ от $\log(n)$ для такой функции, получится $\log(T) = \log(c) + \log(\log n)$ – кривая, наклон которой не постоянен, а стремится к нулю при росте n (поскольку $\log(\log n)$ растёт исчезающе медленно по сравнению с $\log n$). На таком графике логарифмический рост выглядел бы почти горизонтальной линией, легко принимаемой за константу $O(1)$ – то есть `log-log` график в этом случае не только бесполезен, но и вводит в заблуждение, маскируя реальный (пусть и очень медленный) рост. Правильный способ линеаризации для логарифмической зависимости – отложить время не от n и не от $\log(n)$ в логарифмических осях, а от $\log_2 n$ на обычных линейных осях, как и сделано на графике $t(\log_2 n)$: именно тогда $T = c \cdot x + c'$ даёт прямую линию с содержательным наклоном c .

Почему замер выполняется по модулю: Модель RAM, на которой строится теоретическая оценка $\Theta(\log n)$, предполагает, что каждая операция (в частности, умножение) стоит константное время $O(1)$. Это предположение справедливо только для чисел ограниченного размера. В Python целые числа имеют произвольную точность («длинную арифметику»): при возведении в степень без ограничения по модулю величина x на каждом шаге алгоритма возводится в квадрат, и её битовая длина при этом удваивается – то есть сами числа растут экспоненциально по числу бит, а стоимость умножения многобитных чисел растёт вместе с их длиной, а вовсе не остаётся константной. В результате без модуля измеренное время отражало бы не число итераций цикла $(\lfloor \log_2 n \rfloor + 1)$, а скорость роста длины чисел – то есть фактически измерялась бы совсем другая характеристика, сложность длинной арифметики, а не сложность самого алгоритма бинарного возведения в степень.

Применение `% mod` после каждого умножения удерживает все промежуточные значения в пределах фиксированной битовой длины (порядка 30 бит для `POW_MOD = 1_000_000_007`), благодаря чему каждое умножение действительно выполняется за постоянное время – и измеренная зависимость корректно отражает именно логарифмический рост числа итераций, предсказанный теорией.

3. Репрезентативные тесты и инварианты.

Инварианты: для функции `count_equal_pairs`, не имеющей готового эталона среди встроенных функций Python, дополнительно зафиксированы свойства, которые должны выполняться для любого допустимого входа, а не только для отдельных примеров:

1. на массиве, все элементы которого попарно различны, число совпадающих пар всегда равно нулю;

```
assert count_equal_pairs(list(range(50))) == 0
```

2. результат не зависит от порядка элементов массива, поскольку является свойством мультимножества значений, а не их последовательности;

```
rng2 = random.Random(1)
a = [rng2.randint(0, 9) for _ in range(40)]
shuffled = a[:]
rng2.shuffle(shuffled)
assert count_equal_pairs(a) == count_equal_pairs(shuffled)
```

3. результат совпадает с результатом независимого расчёта по формуле $\sum C(c_i, 2)$, где c_i – частота i -го уникального значения в массиве (проверено на 200 случайных входах).

```
def count_equal_pairs_reference(a: list[int]) -> int:
    counts = {}
    for x in a:
        if x in counts:
            counts[x] += 1
        else:
            counts[x] = 1
    total = 0
    for c in counts.values():
        total += c * (c - 1) // 2
    return total

for _ in range(200):
    a = [rng2.randint(0, 9) for _ in range(rng2.randint(1, 60))]
    assert count_equal_pairs(a) == count_equal_pairs_reference(a)
```

4. Сверка с эталонными реализациями

Для функций, имеющих аналог среди встроенных средств Python, результаты сопоставлены с эталонными вычислениями на 200 случайных входах с фиксированным значением seed генератора случайных чисел (для воспроизводимости тестов).

Во всех случаях расхождений между реализованными функциями и эталонными значениями обнаружено не было, что подтверждает корректность реализаций перед переходом к измерению их производительности.

Вывод

В ходе лабораторной работы были реализованы четыре алгоритма трёх классов сложности – линейной (`array_sum`, `array_max`), квадратичной (`count_equal_pairs`) и логарифмической (`binary_row`). Теоретические оценки $\Theta(n)$, $\Theta(n^2)$ и $\Theta(\log n)$, полученные вручную, подтвердились экспериментально: наблюдаемые наклоны на графиках практически совпали с предсказанными значениями (1, 2 и линейная зависимость от $\log_2 n$ соответственно), а единственное заметное отклонение (у квадратичного алгоритма) объяснимо эффектами кэша процессора. Работа показала, что асимптотический анализ, выполненный без запуска кода, достоверно предсказывает реальное поведение алгоритма.